Lecture_5

Dr. Preeti Jagadev

Using readxl to Read an Excel File

```r
#install.packages("readxl")
library("readxl")
library("tidyverse")
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
#Define the URL across multiple lines: make it easier to cut & paste code
part1 <-"http://www2.census.gov/programs-surveys/popest/tables/2010-2011/state/totals/"
part2 <- "nst-est2011-01.xls"
dataFile <- paste0(part1, part2)

#Download the file from the web, into tmpExcelFile
download.file(dataFile, "tmpExcelFile.xls")

#Now read the Excel file into R
testFrame <- read_excel("tmpExcelFile.xls")
```

```
## New names:
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
```

```r
glimpse(testFrame)
```

```
## Rows: 66
## Columns: 5
## $ `table with row headers in column A and column headers in rows 3 through 4. (leading dots indicate
## $ ...2
## $ ...3
## $ ...4
## $ ...5
```

Renaming the stateName Column:

```
#Create a better column name
testFrame$stateName <- pull(testFrame, 1)
testFrame <- testFrame[,-1]

#Remove the '.'
testFrame$stateName <- str_replace(testFrame$stateName,"\\.","")
head(testFrame, 8)
```

```
## # A tibble: 8 x 5
##   ...2      ...3           ...4                                  ...5 stateName
##   <chr>     <chr>          <chr>                                <dbl> <chr>
## 1 <NA>      <NA>           <NA>                                NA      Table 1 ~
## 2 40269     <NA>           Population Estimates (as of July 1) NA      Geograph~
## 3 Census    Estimates Base 2010                                 2.01e3 <NA>
## 4 308745538 308745538      309330219                            3.12e8 United S~
## 5 55317240  55317244       55366108                             5.55e7 Northeast
## 6 66927001  66926987       66976458                             6.72e7 Midwest
## 7 114555744 114555757      114857529                            1.16e8 South
## 8 71945553  71945550       72130124                             7.29e7 West
```

Explanation of the code:

testFrame$stateName <- pull(testFrame, 1): Take the first column of testFrame, and copy it into a new column called stateName. "pull()" is a function from the dplyr package that extracts a column as a vector.

testFrame <- testFrame[,-1]: This removes the first column from the dataset. You just copied it into stateName, so now you're cleaning up and deleting the old version, to avoid having the same column twice.

This code uses the str_replace() function. It replaces any period (.) in the stateName with nothing (i.e., deletes it). Example: "N.Y." → "NY"

head(testFrame, 8): Show the first 8 rows of the testFrame data.

```
#Remove the first eight rows
testFrame <- slice(testFrame, -1:-8)
```

Continue Cleaning the DataFrame

```
#Remove the bottom lines
testFrame <- slice(testFrame, -52:-58)

#Rename the columns
testFrame <- testFrame %>% rename(april10census = '...2')
testFrame <- testFrame %>% rename(april10base = '...3')
testFrame <- testFrame %>% rename(july10pop = '...4')
testFrame <- testFrame %>% rename(july11pop = '...5')

#Make the populations numeric
testFrame <- testFrame %>%
          mutate_at(vars(april10census,april10base,july10pop,july11pop),
                                        as.numeric)
```

Explanation of the code:

STEP 1: Renaming Columns

testFrame <- testFrame %>% rename(april10census = '...2')

testFrame <- testFrame %>% rename(april10base = '...3')

testFrame <- testFrame %>% rename(july10pop = '...4')

testFrame <- testFrame %>% rename(july11pop = '...5')

Here, each line is renaming one column. For example:

rename(april10census = '...2')

says: take the column called ...2 and rename it to april10census.

STEP 2: Convert Columns to Numeric

testFrame <- testFrame %>% mutate_at(vars(april10census, april10base, july10pop, july11pop), as.numeric):

mutate_at() lets you apply a function to multiple columns at once.

vars(...) lists the columns you want to convert.

as.numeric changes values like "12345" (a string) into 12345 (a number).

Why is this needed?

Sometimes when you import data, especially from Excel or text files, numbers are read as text. For example, "1000" instead of 1000.

If numbers are stored as text, you can't do math with them, so converting to numeric is important.

The Cleaned DataFrame

```r
#Look at the first few rows
head(testFrame)
```

```
## # A tibble: 6 x 5
##   april10census april10base july10pop july11pop stateName
##           <dbl>       <dbl>     <dbl>     <dbl> <chr>
## 1       4779736     4779735   4785401   4802740 Alabama
## 2        710231      710231    714146    722718 Alaska
## 3       6392017     6392013   6413158   6482505 Arizona
## 4       2915918     2915921   2921588   2937979 Arkansas
## 5      37253956    37253956  37338198  37691912 California
## 6       5029196     5029196   5047692   5116796 Colorado
```

Using group_by

```r
#Remove the District of Columbia column
testFrame <- testFrame %>%  filter(stateName != "District of Columbia")

#Add the region for each state
testFrame <- testFrame %>% add_column(region=state.region)

#Calculate the region mean
testFrame %>% group_by(region) %>%
           summarise( regionMean = mean(july11pop) )
```

```
## # A tibble: 4 x 2
##   region        regionMean
##   <fct>              <dbl>
```

```
## 1 Northeast       6169066.
## 2 South           7214296.
## 3 North Central   5596570.
## 4 West            5604981.
```

Step 1: Remove the "District of Columbia" Row

testFrame <- testFrame %>%
filter(stateName != "District of Columbia")

The filter() function keeps only the rows that meet a condition.

stateName != "District of Columbia" means: keep all rows except the one where the state is "District of Columbia".

So, you're removing the District of Columbia from the dataset.

Step 2: Add a Region Column:

testFrame <- testFrame %>%
add_column(region = state.region)

Adds a new column to your table called region.

state.region is a built-in R vector that gives the U.S. region for each of the 50 states, like "Northeast", "South", "Midwest", or "West".

Example: If stateName = "California", then region = "West" If stateName = "New York", then region = "Northeast"

Step 3: Calculate the Mean Population for Each Region:

testFrame %>% group_by(region) %>%
summarise(regionMean = mean(july11pop))

What This Does: group_by(region): This groups all the states by their region, so all "West" states together, all "South" states together, etc.

summarise(regionMean = mean(july11pop)): For each region, it calculates the mean of the July 2011 population.

```r
#Store the region mean as a new column
testFrame <-testFrame %>%  group_by(region) %>% mutate(regionMean = mean(july11pop))

#Look at the updated dataframe
head(testFrame, 2)
```

```
## # A tibble: 2 x 7
## # Groups:   region [2]
##   april10census april10base july10pop july11pop stateName region regionMean
##           <dbl>       <dbl>     <dbl>     <dbl> <chr>     <fct>       <dbl>
## 1       4779736     4779735   4785401   4802740 Alabama   South    7214296.
## 2        710231      710231    714146    722718 Alaska    West     5604981.
```

Step 1: Store the Region Mean in a New Column:

testFrame <- testFrame %>% group_by(region) %>% mutate(regionMean = mean(july11pop))

group_by(region): Groups the data by the region each state belongs to (like Northeast, South, etc.).

mutate(regionMean = mean(july11pop)): Within each region group, it calculates the average population for July 2011, and adds that value as a new column called regionMean.

But here's the key point: Instead of creating a separate summary table (like in summarise()), mutate() keeps the original data and adds the region mean to each row.

So, if "California" is in the "West" region and the average July 2011 population of all western states is 7 million, California's row will now include:

region = "West" regionMean = 7000000 And every other Western state will have the same regionMean value.

Use sqldf for Developing Queries:

```r
install.packages("sqldf")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.4'
## (as 'lib' is unspecified)
```

```r
library(sqldf)
```

```
## Loading required package: gsubfn

## Loading required package: proto

## Warning in fun(libname, pkgname): couldn't connect to display ":0"

## Loading required package: RSQLite
```

```r
sqldf('select mpg from mtcars')
```

```
##     mpg
## 1  21.0
## 2  21.0
## 3  22.8
## 4  21.4
## 5  18.7
## 6  18.1
## 7  14.3
## 8  24.4
## 9  22.8
## 10 19.2
## 11 17.8
## 12 16.4
## 13 17.3
## 14 15.2
## 15 10.4
## 16 10.4
## 17 14.7
## 18 32.4
## 19 30.4
## 20 33.9
## 21 21.5
## 22 15.5
## 23 15.2
## 24 13.3
## 25 19.2
## 26 27.3
## 27 26.0
## 28 30.4
## 29 15.8
## 30 19.7
```

```
## 31 15.0
## 32 21.4
```

```r
sqldf('select AVG(mpg) AS avg_mpg_4cyl FROM mtcars WHERE cyl = 4')
```

```
##   avg_mpg_4cyl
## 1     26.66364
```

The sqldf package lets you use SQL language on data frames in R. You only need to install once, but library(sqldf) is needed every time you restart R.

sqldf('SELECT mpg FROM mtcars') runs an SQL query:

SELECT mpg → Get the mpg column (miles per gallon).

FROM mtcars → Use the built-in mtcars data frame in R

sqldf('SELECT AVG(mpg) AS avg_mpg_4cyl FROM mtcars WHERE cyl = 4')

SELECT AVG(mpg): Calculates the average mpg

FROM mtcars: Using the mtcars dataset

WHERE cyl = 4: But only for rows (cars) where cyl = 4 (4-cylinder cars)

AS avg_mpg_4cyl: Gives the result column a nice name (avg_mpg_4cyl)

What's Happening Behind the Scenes?

You are:

Accessing an R dataset (mtcars)

Writing SQL code like you would in a database

Letting sqldf handle the connection between R and SQL

Getting results back as an R data frame

Final Output: This small bit of code gives you powerful SQL-like querying without leaving R.

JSON Data Access:

```r
library(RCurl)
```

```
##
## Attaching package: 'RCurl'

## The following object is masked from 'package:tidyr':
##
##     complete
```

```r
library(jsonlite)
```

```
##
## Attaching package: 'jsonlite'

## The following object is masked from 'package:purrr':
##
##     flatten
```

```r
bikeURL <-
     'https://gbfs.citibikenyc.com/gbfs/en/station_status.json'

apiResult <- getURL(bikeURL)
```

```r
results <- fromJSON(apiResult)
str(results)
```

```
## List of 4
##  $ data        :List of 1
##   ..$ stations:'data.frame': 2236 obs. of  14 variables:
##   .. ..$ eightd_has_available_keys: logi [1:2236] FALSE FALSE FALSE FALSE FALSE FALSE ...
##   .. ..$ num_bikes_disabled       : int [1:2236] 0 0 0 0 0 0 1 0 0 0 ...
##   .. ..$ num_ebikes_available     : int [1:2236] 0 0 0 0 0 0 2 5 3 6 ...
##   .. ..$ is_installed             : int [1:2236] 0 0 0 0 0 0 1 1 1 1 ...
##   .. ..$ num_bikes_available      : int [1:2236] 0 0 0 0 0 0 30 18 3 16 ...
##   .. ..$ station_id               : chr [1:2236] "cc5f0e68-dd5f-4db1-81c7-04a4d6674fba" "576abdc1-98
##   .. ..$ is_returning             : int [1:2236] 0 0 0 0 0 0 1 1 1 1 ...
##   .. ..$ last_reported            : int [1:2236] 1751977116 1750773080 1750767148 1750163715 1750444
##   .. ..$ num_docks_available      : int [1:2236] 0 0 0 0 0 0 9 14 18 5 ...
##   .. ..$ num_docks_disabled       : int [1:2236] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ legacy_id                : chr [1:2236] "4131" "4772" "4774" "3731" ...
##   .. ..$ is_renting               : int [1:2236] 0 0 0 0 0 0 1 1 1 1 ...
##   .. ..$ num_scooters_unavailable : int [1:2236] NA NA NA NA NA NA 0 0 0 0 ...
##   .. ..$ num_scooters_available   : int [1:2236] NA NA NA NA NA NA 0 0 0 0 ...
##  $ last_updated: int 1752026483
##  $ ttl         : int 60
##  $ version     : chr "1.1"
```

What This Code Is Doing: You're using R to connect to a live bike-sharing website, get real-time info about CitiBike stations in New York City, and load that info into R so you can explore or analyze it.

RCurl: Helps R connect to websites or APIs.

jsonlite: Helps R understand JSON data, which is a format used by most modern APIs.

bikeURL <- 'https://gbfs.citibikenyc.com/gbfs/en/station_status.json' You are storing the web address of the CitiBike NYC station status API into a variable called bikeURL. This URL gives you live data about every CitiBike station, like how many bikes are available, how many docks are empty, etc.

apiResult <- getURL(bikeURL)

This uses getURL() from the RCurl package to go to that web address and download the raw JSON text from it. Think of it like copy-pasting the website's data into R.

results <- fromJSON(apiResult) Now you're converting that raw text into a list or data frame that R can work with, using fromJSON().

So now you can use this results variable like a regular R object, it contains all the bike station info.

```r
#See the data generated
stations <- results$data$stations
glimpse(stations)
```

```
## Rows: 2,236
## Columns: 14
## $ eightd_has_available_keys <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
## $ num_bikes_disabled        <int> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1,~
## $ num_ebikes_available      <int> 0, 0, 0, 0, 0, 0, 2, 5, 3, 6, 2, 19, 1, 5, 0~
## $ is_installed              <int> 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ num_bikes_available       <int> 0, 0, 0, 0, 0, 0, 30, 18, 3, 16, 16, 20, 18,~
## $ station_id                <chr> "cc5f0e68-dd5f-4db1-81c7-04a4d6674fba", "576~
## $ is_returning              <int> 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ last_reported             <int> 1751977116, 1750773080, 1750767148, 17501637~
```

```
## $ num_docks_available       <int> 0, 0, 0, 0, 0, 0, 9, 14, 18, 5, 2, 0, 3, 12,~
## $ num_docks_disabled        <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ legacy_id                 <chr> "4131", "4772", "4774", "3731", "18145480674~
## $ is_renting                <int> 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ num_scooters_unavailable  <int> NA, NA, NA, NA, NA, NA, 0, 0, 0, 0, 0, 0, 0,~
## $ num_scooters_available    <int> NA, NA, NA, NA, NA, NA, 0, 0, 0, 0, 0, 0, 0,~
```

You're saying:

"Hey R, go inside results, then inside data, and then grab the thing called stations."

You save that chunk into a new variable called stations.

```r
#Keep a subset of the columns
colsToKeep <- c('num_bikes_disabled','num_docks_disabled',
     'station_id', 'num_ebikes_available', 'num_bikes_available',
     'num_docks_available')

#Both of these commands do the same thing
stations1 <- stations[,colsToKeep]
stations2 <- stations %>% select(all_of(colsToKeep))
str(stations2)
```

```
## 'data.frame':    2236 obs. of  6 variables:
##  $ num_bikes_disabled  : int  0 0 0 0 0 0 1 0 0 0 ...
##  $ num_docks_disabled  : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ station_id          : chr  "cc5f0e68-dd5f-4db1-81c7-04a4d6674fba" "576abdc1-986e-48b0-bde2-051a13
##  $ num_ebikes_available: int  0 0 0 0 0 0 2 5 3 6 ...
##  $ num_bikes_available : int  0 0 0 0 0 0 30 18 3 16 ...
##  $ num_docks_available : int  0 0 0 0 0 0 9 14 18 5 ...
```

```r
#Average number of empty docks
mean(stations2$num_docks_available)
```

```
## [1] 13.41279
```

```r
#Average number of available bikes
mean(stations2$num_bikes_available)
```

```
## [1] 15.70394
```

```r
#How many stations have a bike available (could have used 'filter')
bikesAvailDF <- stations2[stations2$num_bikes_available>0,]
nrow(bikesAvailDF)
```

```
## [1] 2120
```

Why do you think citibike made the data available via JSON (why not CSV)?

Reason:

1. JSON works better for nested, structured data: CitiBike's data isn't just rows and columns, it has multiple layers (like a list of stations, and inside each station: status, ID, location, etc). JSON handles this naturally. CSV, on the other hand, is just a flat table, no nesting, no hierarchy.
2. JSON is the standard for web APIs Almost all real-time web services use JSON because it's: Lightweight, easy for computers to read and write, supported by most programming languages (R, Python, JavaScript, etc).
3. Live/Real-Time Data Is Better in JSON CitiBike data is updated constantly (every few seconds). JSON is built for live data streaming and works well with servers. CSV is usually used for static, exported

files (not real-time feeds).

4. More Data Types Supported JSON supports types like:numbers, strings, booleans (true/false), nested objects, and arrays. CSV only supports text, and everything is flattened.